BASIC SALESFORCE QUESTIONS (1-20)

Q1. What is Salesforce?

Answer

Salesforce is a cloud-based CRM (Customer Relationship Management) platform used by companies to manage customers, sales, marketing, and support activities.

Instead of storing customer data in Excel sheets, companies use Salesforce to keep everything in one place.

Example

A company receives 100 customer inquiries daily.

Salesforce helps them:

- Store customer information
- Track sales
- Follow up customers
- Generate reports

Q2. What is CRM?

Answer

CRM stands for Customer Relationship Management.

It is a system used to manage interactions with customers.

Example

Suppose Amazon wants to remember:

- Customer Name
- Email
- Purchase History
- Complaints

CRM helps store and manage this information.

Q3. What is Sales Cloud?

Answer

Sales Cloud is a Salesforce product used for managing the sales process.

It helps sales teams:

- Track Leads
- Manage Opportunities
- Generate Quotes
- Close Deals

Real Example

Customer wants software.

Sales Team:

Lead → Opportunity → Quote → Order

This entire process happens in Sales Cloud.

Q4. What is a Lead?

Answer

A Lead is a person or company that has shown interest in your product but has not yet been qualified.

Example

Someone fills a website form:

Name: Megha
Email: megha@gmail.com

This becomes a Lead.

Q5. What is Lead Qualification?

Answer

Lead Qualification means checking whether the lead is worth pursuing.

Sales team checks:

- Budget
- Requirement

- Decision-making power

Example

Customer wants product and has budget.

Lead is qualified.

Q6. What happens during Lead Conversion?

Answer

When a Lead is qualified, Salesforce converts it into:

- Account
- Contact
- Opportunity (optional)

Flow

Lead
↓
Account
↓
Contact
↓
Opportunity

Interview Question

Can we convert Lead without Opportunity?

Yes.

Q7. What is an Account?

Answer

Account represents a company or organization.

Examples

- Google
- Microsoft
- Amazon

Each company is stored as an Account.

Q8. What is a Contact?

Answer

Contact is a person associated with an Account.

Example

Account = Amazon

Contact = John Smith

Job Title = Manager

Q9. Difference Between Account and Contact?

Answer

Account	Contact
Company	Person
Amazon	John
Google	David

Example

Amazon = Account

HR Manager = Contact

Q10. What is Opportunity?

Answer

Opportunity represents a potential deal or sale.

Example

Amazon wants software worth ₹10,00,000.

This deal becomes an Opportunity.

Q11. What are Opportunity Stages?

Answer

Opportunity passes through stages.

Common stages:

- Prospecting
- Qualification
- Proposal
- Negotiation
- Closed Won
- Closed Lost

Example

Customer requested demo.

Stage = Qualification

Q12. Difference Between Closed Won and Closed Lost?

Answer

Closed Won:

Customer purchased product.

Closed Lost:

Customer rejected product.

Q13. What is a Campaign?

Answer

Campaign is a marketing activity.

Examples

- Email Campaign
- Facebook Ads
- Webinar

- Event

Purpose:

Generate Leads.

Q14. What is Campaign Member?

Answer

A person associated with a campaign.

Example

Webinar Campaign

500 people attended.

All become Campaign Members.

Q15. What is Product?

Answer

Anything sold by a company.

Example

- Laptop
- Software
- Mobile

Q16. What is Price Book?

Answer

Price Book stores product prices.

Example

Laptop

Standard Price = ₹50,000

Festival Price = ₹45,000

Different prices stored in different Price Books.

Q17. What is Opportunity Product?

Answer

Product added to an Opportunity.

Example

Opportunity:

Amazon Purchase

Products:

- Laptop x10
- Mouse x20

These are Opportunity Products.

Q18. What is Quote?

Answer

Quote is a price proposal sent to customer.

Example

Laptop = ₹50,000
Quantity = 10

Total = ₹5,00,000

This document is a Quote.

Q19. What is Quote Line Item?

Answer

Each product inside a Quote.

Example

Quote:

Laptop

Mouse

Keyboard

Each item is a Quote Line Item.

Q20. What is Contract?

Answer

Contract is a legal agreement between customer and company.

Example

1-year software subscription agreement.

INTERMEDIATE SALESFORCE QUESTIONS (21-40)

Q21. Difference Between Lead and Opportunity?

Answer

Lead = Potential customer.

Opportunity = Potential sale.

Lead comes before Opportunity.

Q22. Can One Account Have Multiple Contacts?

Answer

Yes.

Example

Amazon

- HR Manager
- IT Manager
- CEO

All are Contacts.

Q23. Can One Account Have Multiple Opportunities?

Answer

Yes.

Example

Amazon buys:

- Software
- Hardware
- Consulting

Three Opportunities.

Q24. Why Do We Need Price Books?

Answer

Different customers may receive different pricing.

Example

Standard Customer = ₹50,000

Partner Customer = ₹45,000

Q25. What is Forecasting?

Answer

Predicting future sales revenue.

Example

Current Opportunities:

₹5 Lakh

₹10 Lakh

₹20 Lakh

Forecast = ₹35 Lakh

Q26. What is Pipeline?

Answer

All open Opportunities together.

Example

Sales manager checks upcoming revenue.

Q27. What is Sales Process?

Answer

Stages through which Opportunity moves.

Example:

Prospecting → Proposal → Closed Won

Q28. What is Page Layout?

Answer

Controls fields visible on record page.

Example

Manager sees extra fields.

Sales Rep sees limited fields.

Q29. What is Record Type?

Answer

Used when same object requires different business processes.

Example

Opportunity:

- Domestic
- International

Q30. What is Validation Rule?

Answer

Used to prevent invalid data.

Example

Phone Number cannot be blank.

Q31. What is Workflow?

Answer

Automates actions.

Example

When Opportunity is Won

→ Send Email

Q32. What is Process Builder?

Answer

Automation tool.

Can update records and create tasks automatically.

Q33. What is Flow?

Answer

Most powerful Salesforce automation tool.

Can:

- Create records
- Update records
- Delete records

Interview Tip

Flow is replacing Workflow and Process Builder.

Q34. What is Approval Process?

Answer

Used when records need approval.

Example

Discount > 30%

Manager approval required.

Q35. What is Report?

Answer

Displays business data.

Example

All Open Opportunities.

Q36. What is Dashboard?

Answer

Visual representation of reports.

Example

Sales Chart

Revenue Chart

Pipeline Chart

Q37. Difference Between Role and Profile?

Answer

Profile:

What user can do.

Role:

What data user can see.

Q38. What is Sharing Rule?

Answer

Used to share records with other users.

Q39. What is Lookup Relationship?

Answer

Loose relationship between two objects.

Example

Employee → Department

Q40. What is Master-Detail Relationship?

Answer

Strong relationship.

Child depends on Parent.

Example

Order → Order Item

Delete Order

Order Items also deleted.

SALESFORCE PRACTICAL TASKS (10)

Task 1

Create Lead Object Record

Solution

- Open Leads Tab
- Click New
- Enter Name
- Save

Task 2

Convert Lead

Steps

- Open Lead
- Click Convert
- Create Account
- Create Contact
- Create Opportunity

Task 3

Create Opportunity

Steps

- Open Account
- New Opportunity
- Enter Amount
- Select Stage

Task 4

Create Product

Steps

- Product Tab
- New Product
- Enter Name
- Save

Task 5

Add Product to Opportunity

Steps

- Open Opportunity
- Add Product

- Select Price Book
- Save

Task 6

Create Quote

Steps

- Open Opportunity
- New Quote
- Add Products
- Save

Task 7

Create Report

Steps

- Reports Tab
- New Report
- Opportunities Report
- Save

Task 8

Create Dashboard

Steps

- Dashboard Tab
- New Dashboard
- Add Report Components

Task 9

Create Validation Rule

Requirement

Phone cannot be blank.

Formula

ISBLANK(Phone)

Error:

Phone is required.

Task 10

Create Flow

Requirement

When Opportunity becomes Closed Won

Automatically create Task.

Solution

Record Triggered Flow
↓
Opportunity Updated
↓
Stage = Closed Won
↓
Create Task

CORE NODE.JS QUESTIONS (1-20)

Q1. What is Node.js?

Answer

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser.

Before Node.js:

- JavaScript only worked in browsers.

After Node.js:

- We can create APIs.
- We can connect databases.
- We can build backend applications.

Example

React Frontend
↓
Node.js Server
↓
MongoDB

Node.js receives requests, processes them, and sends responses.

Q2. Why Do We Use Node.js?

Answer

Node.js is used because it is:

- Fast
- Lightweight
- Scalable
- Event-driven

Example

Applications like:

- Netflix
- PayPal
- Uber

handle thousands of users simultaneously.

Q3. What is V8 Engine?

Answer

V8 Engine is Google's JavaScript engine used by Node.js.

It converts JavaScript into machine code.

Interview Point

Node.js is fast because of the V8 Engine.

Q4. What is Event Loop?

Answer

The Event Loop helps Node.js perform non-blocking operations.

Example

Suppose:

```
setTimeout(()=>{
 console.log("Hello")
},2000)

console.log("World")
```

Output:

```
World
Hello
```

Node doesn't wait for timeout.

Q5. What is Non-Blocking I/O?

Answer

Node.js doesn't stop execution while waiting for operations.

Example

Reading a file:

fs.readFile()

Node continues executing other tasks.

Q6. Difference Between Synchronous and Asynchronous?

Synchronous

Runs one by one.

```
console.log("A")
console.log("B")
```

Output:

A
B

Asynchronous

Doesn't wait.

```
setTimeout(()=>{
 console.log("A")
},1000)

console.log("B")
```

Output:

B
A

Q7. What is Callback?

Answer

A function passed as an argument to another function.

Example

```
function greet(callback){
 callback()
}
```

Q8. What is Callback Hell?

Answer

Nested callbacks making code difficult to read.

Example

```
login(()=>{
 getUser(()=>{
```

```
  getOrders(()=>{
  })
 })
})
```

Solution:

- Promises
- Async/Await

Q9. What is a Promise?

Answer

Promise handles asynchronous operations.

States:

- Pending
- Resolved
- Rejected

Example

```
fetchData()
.then()
.catch()
```

Q10. What is Async/Await?

Answer

Cleaner way to work with Promises.

Example

```
async function getData(){
 const data = await fetch(url)
}
```

Q11. What is npm?

Answer

npm = Node Package Manager.

Used to install packages.

Example

npm install express

Q12. What is package.json?

Answer

Stores project information.

Contains:

- Project name
- Version
- Dependencies
- Scripts

Example

```
{
 "name":"node-app",
 "version":"1.0.0"
}
```

Q13. What is a Module?

Answer

A module is reusable code.

Example

module.exports = add

Q14. What is CommonJS?

Answer

Module system used by Node.js.

Import

const fs = require("fs")

Export

module.exports = add

Q15. What is require()?

Answer

Used to import modules.

Example

const express = require("express")

Q16. What is process object?

Answer

Provides information about running Node process.

Example

console.log(process.env)

Used for environment variables.

Q17. What is Buffer?

Answer

Temporary memory used to store binary data.

Example

Files
Images
Videos

Q18. What is fs Module?

Answer

Used to work with files.

Example

fs.readFile()
fs.writeFile()

Q19. What is path Module?

Answer

Works with file paths.

Example

path.join()

Creates safe paths.

Q20. What are Streams?

Answer

Streams process large data in chunks.

Example

Video Streaming

Instead of loading 2GB file completely,
Node sends chunks.

ADVANCED NODE.JS QUESTIONS (21-40)

Q21. What is Single Threaded?

Answer

Node.js runs JavaScript in one main thread.

However it can handle many requests through Event Loop.

Q22. If Node is Single Threaded, How Does It Handle Multiple Users?

Answer

Using:

- Event Loop

- Callback Queue
- Thread Pool

This is one of the most asked questions.

Q23. What is Thread Pool?

Answer

Provided by libuv.

Used for:

- File operations
- Cryptography
- DNS lookup

Q24. What is libuv?

Answer

Library responsible for:

- Event Loop
- Async I/O
- Thread Pool

Q25. What is Global Object?

Answer

Like window in browser.

Node uses:

global

Q26. What is Middleware?

Answer

Function executed before request reaches route.

Example:

app.use(express.json())

(More detailed in Express section.)

Q27. What is Environment Variable?

Answer

Stores sensitive information.

Example

PORT=3000
JWT_SECRET=abc123

Q28. Why Use dotenv?

Answer

Loads environment variables from .env file.

require('dotenv').config()

Q29. What is process.env?

Answer

Accesses environment variables.

process.env.PORT

Q30. What is Memory Leak?

Answer

Memory is allocated but never released.

Result:

- Slow application
- Crash

Q31. What is Clustering?

Answer

Creates multiple Node processes.

Uses all CPU cores.

Q32. What is Child Process?

Answer

Runs another process from Node.

Example

```
const {spawn} = require('child_process')
```

Q33. What is Error-First Callback?

Example

```
fs.readFile(file,(err,data)=>{
 if(err){
  return console.log(err)
 }
})
```

First parameter is always error.

Q34. What is Event Emitter?

Answer

Allows custom events.

Example

```
eventEmitter.emit()
eventEmitter.on()
```

Q35. What is REPL?

Answer

Read Eval Print Loop.

Interactive Node shell.

node

Q36. Difference Between setTimeout and setInterval?

setTimeout

Runs once.

setInterval

Runs repeatedly.

Q37. What is Hot Reloading?

Answer

Restart app automatically after code changes.

Tool:

nodemon

Q38. What is package-lock.json?

Answer

Stores exact dependency versions.

Ensures same installation on every machine.

Q39. What is Dependency Injection?

Answer

Providing dependencies from outside rather than creating inside.

Makes testing easier.

Q40. Why is Node.js Popular for APIs?

Answer

Because it is:

- Fast
- Lightweight
- Asynchronous
- Great for JSON

NODE.JS PRACTICAL TASKS (10)

Task 1

Create HTTP Server

Expected

```
const http = require('http')

http.createServer((req,res)=>{
 res.end("Hello")
}).listen(3000)
```

Task 2

Read File

Expected

```
fs.readFile('test.txt','utf8',(err,data)=>{
 console.log(data)
})
```

Task 3

Write File

Expected

```
fs.writeFile('demo.txt','hello',()=>{})
```

Task 4

Create Custom Module

add.js

```
function add(a,b){
 return a+b
}

module.exports = add
```

Task 5

Import Module

```
const add = require('./add')
```

Task 6

Create Promise

```
const promise = new Promise((resolve,reject)=>{
 resolve("Success")
})
```

Task 7

Convert Callback to Async/Await

Interviewers love this.

Task 8

Read Environment Variable

```
console.log(process.env.PORT)
```

Task 9

Create Event Emitter

```
emitter.on()
emitter.emit()
```

Task 10

Handle Errors Properly

```
try{
 await getData()
}
catch(err){
 console.log(err)
}
```

CORE EXPRESS.JS QUESTIONS (1-20)

Q1. What is Express.js?

Answer

Express.js is a lightweight web framework built on Node.js.

It helps us create:

- APIs
- Web Applications
- Backend Servers

Without Express:

const http = require('http')

A lot of code is required.

With Express:

```
const express = require('express')
const app = express()
```

Much easier.

Q2. Why Do We Use Express.js?

Answer

Express simplifies backend development.

Benefits:

- Routing
- Middleware
- API Creation
- Error Handling

Q3. What is app.listen()?

Answer

Starts the server and listens for requests.

```
app.listen(3000,()=>{
 console.log("Server Running")
})
```

Q4. What is Routing?

Answer

Routing determines which function runs for a specific URL.

Example:

```
app.get('/home',(req,res)=>{
 res.send("Home")
})
```

Q5. What is Middleware?

Answer

Middleware is a function that runs between:

```
Request
  ↓
Middleware
  ↓
Route Handler
```

↓
Response

Q6. What is req Object?

Answer

Contains client request data.

Examples:

req.body
req.params
req.query
req.headers

Q7. What is res Object?

Answer

Used to send response back.

Example:

res.send()
res.json()
res.status()

Q8. Difference Between res.send() and res.json()?

res.send()

Can send:

- String
- HTML
- Object

res.json()

Always sends JSON.

res.json({name:"Megha"})

Q9. What is express.json()?

Answer

Parses incoming JSON request body.

app.use(express.json())

Without it:

req.body

becomes undefined.

Q10. What is app.use()?

Answer

Used to register middleware.

Example:

app.use(express.json())

Q11. Difference Between app.use() and app.get()?

app.use()

Runs middleware.

app.get()

Handles GET request.

Q12. What is GET Request?

Answer

Fetches data.

app.get('/users')

Q13. What is POST Request?

Answer

Creates new data.

app.post('/users')

Q14. What is PUT Request?

Answer

Updates complete record.

Q15. What is PATCH Request?

Answer

Updates selected fields.

Q16. What is DELETE Request?

Answer

Deletes data.

app.delete('/users/:id')

Q17. What is REST API?

Answer

A standard way of creating APIs using:

- GET
- POST
- PUT
- PATCH
- DELETE

Q18. What is Status Code?

Common Codes

200 OK
201 Created
400 Bad Request

401 Unauthorized
404 Not Found
500 Internal Server Error

Q19. What is CORS?

Answer

Cross-Origin Resource Sharing.

Allows frontend and backend communication.

Example:

app.use(cors())

Q20. What is Express Router?

Answer

Used to separate routes.

Example:

const router = express.Router()

ADVANCED EXPRESS QUESTIONS (21-40)

Q21. What is Route Parameter?

Example

/users/:id

Request:

/users/101

Access:

req.params.id

Q22. What is Query Parameter?

Example

/users?age=25

Access:

req.query.age

Q23. Difference Between Params and Query?

Params

Mandatory

/users/10

Query

Optional

/users?page=1

Q24. What is Custom Middleware?

Example

```
const logger=(req,res,next)=>{
 console.log(req.url)
 next()
}
```

Q25. What Happens If next() Is Not Called?

Answer

Request gets stuck.

Response never reaches route.

Very common interview question.

Q26. What is Error Middleware?

Example

```
app.use((err,req,res,next)=>{
 res.status(500).json({
```

```
  error:err.message
 })
})
```

Q27. What is Route Chaining?

Example

```
router
.route('/user')
.get()
.post()
.put()
```

Q28. What is Express Static Middleware?

Example

app.use(express.static('public'))

Serves static files.

Q29. What is Morgan?

Answer

Logging middleware.

app.use(morgan('dev'))

Q30. What is Helmet?

Answer

Security middleware.

app.use(helmet())

Q31. What is Compression Middleware?

Answer

Reduces response size.

Improves performance.

Q32. What is Rate Limiting?

Answer

Restricts requests.

Example:

100 requests per minute.

Q33. What is JWT Authentication?

Flow

Login
↓
Generate JWT
↓
Send Token
↓
Verify Token

Q34. What is JWT Token?

Answer

JSON Web Token used for authentication.

Contains:

```
{
 id:"123"
}
```

Q35. Difference Between Authentication and Authorization?

Authentication

Who are you?

Authorization

What can you access?

Q36. What is bcrypt?

Answer

Used for password hashing.

Never store plain passwords.

Q37. How Do You Protect Routes?

Example

app.get('/profile',authMiddleware)

Q38. What is MVC Architecture?

Structure

Model
View
Controller

Most Express projects use MVC.

Q39. How Do You Organize Express Projects?

Folder Structure

controllers
routes
models
middleware
config

Q40. Difference Between Monolithic and Microservices?

Monolithic

Everything in one app.

Microservices

Multiple small services.

MIDDLEWARE SCENARIOS

Scenario 1

Log Every Request

```
const logger=(req,res,next)=>{
 console.log(req.method)
 next()
}
```

Scenario 2

Check API Key

```
const apiKey=(req,res,next)=>{
 if(req.headers.apikey==="123"){
   next()
 }
 else{
   res.status(401).send("Unauthorized")
 }
}
```

Scenario 3

Check Admin Access

if(user.role==="admin")

Allow request.

Otherwise reject.

JWT SCENARIOS

Scenario 1: Login API

User sends:

```
{
 "email":"test@gmail.com",
 "password":"123"
}
```

Server:

```
const token = jwt.sign(
 {id:user._id},
 secret
)
```

Returns token.

Scenario 2: Protected Route

```
app.get(
 '/profile',
 verifyToken,
 controller
)
```

Middleware verifies JWT.

Scenario 3: Invalid Token

Response:

401 Unauthorized

ROUTING SCENARIOS

Scenario 1

Get User By ID

GET /users/101

Access:

req.params.id

Scenario 2

Search User

/users?name=megha

Access:

req.query.name

Scenario 3

Nested Routes

/users/:id/orders

Example:

/users/101/orders

EXPRESS CODING TASKS (10)

Task 1

Create Express Server

```
const express=require('express')
const app=express()

app.listen(3000)
```

Task 2

Create GET API

```
app.get('/users',(req,res)=>{
 res.send("Users")
})
```

Task 3

Create POST API

```
app.post('/users',(req,res)=>{
 res.send("Created")
})
```

Task 4

Use express.json()

app.use(express.json())

Task 5

Create Custom Middleware

```
const logger=(req,res,next)=>{
 console.log("Request")
 next()
}
```

Task 6

Create Route Parameters

/users/:id

Task 7

Create Query Parameters

/users?page=1

Task 8

Create JWT Login API

Generate token using:

jwt.sign()

Task 9

Create Protected Route

Use:

jwt.verify()

Task 10

Create Global Error Handler

```
app.use((err,req,res,next)=>{
 res.status(500).json({
   message:err.message
 })
})
```

CORE MONGODB QUESTIONS (1-20)

Q1. What is MongoDB?

Answer

MongoDB is a NoSQL database that stores data in JSON-like documents.

Unlike SQL databases, MongoDB does not store data in rows and tables.

Example

```
{
  "name":"Megha",
  "age":25
}
```

This is called a document.

Q2. What is NoSQL?

Answer

NoSQL means "Not Only SQL".

Data is stored in flexible documents instead of fixed tables.

Benefits

- Faster development
- Flexible structure
- Easy scaling

Q3. What is a Database?

Answer

A database is a container for collections.

Example

HotelDB

Contains:

- Users Collection
- Menu Collection
- Orders Collection

Q4. What is a Collection?

Answer

A collection is similar to a table in SQL.

Example

Users

Collection contains multiple user documents.

Q5. What is a Document?

Answer

A document is a single record.

Example

```
{
 "name":"Megha",
```

"age":25
}

Q6. What is BSON?

Answer

MongoDB stores data internally as BSON.

BSON = Binary JSON

Supports:

- Date
- ObjectId
- Binary Data

Q7. What is ObjectId?

Answer

MongoDB automatically creates a unique id.

Example:

665f9e8b3f1a123456789abc

Q8. Difference Between SQL and MongoDB?

SQL	MongoDB
Tables	Collections
Rows	Documents
Schema Fixed	Flexible
Joins	Populate

Q9. What is MongoDB Compass?

Answer

GUI tool used to view database data visually.

Q10. What is Mongo Shell?

Answer

Command-line interface for MongoDB.

Q11. What is CRUD?

Answer

CRUD means:

- Create
- Read
- Update
- Delete

Q12. How to Insert Data?

Example

```
db.users.insertOne({
 name:"Megha"
})
```

Q13. How to Read Data?

Example

```
db.users.find()
```

Q14. How to Update Data?

Example

```
db.users.updateOne()
```

Q15. How to Delete Data?

Example

db.users.deleteOne()

Q16. What is find()?

Answer

Returns multiple documents.

User.find()

Q17. What is findOne()?

Answer

Returns first matching document.

User.findOne()

Q18. What is Projection?

Answer

Used to select specific fields.

Example

```
User.find({},{
 name:1
})
```

Returns only name.

Q19. What is Sorting?

Example

```
User.find().sort({
 age:-1
})
```

Descending order.

Q20. What is Limit?

Example

User.find().limit(5)

Returns 5 records only.

ADVANCED MONGODB & MONGOOSE QUESTIONS (21-40)

Q21. What is Mongoose?

Answer

Mongoose is an ODM (Object Data Modeling) library.

It helps Node.js communicate with MongoDB.

Q22. What is a Schema?

Example

```
const personSchema =
new mongoose.Schema({
 name:String,
 age:Number
})
```

Defines data structure.

Q23. What is a Model?

Example

```
const Person =
mongoose.model(
 'Person',
 personSchema
)
```

Used to perform database operations.

Q24. Difference Between Schema and Model?

Schema

Structure of data.

Model

Used to interact with collection.

Q25. What is Validation?

Example

```
name:{
 required:true
}
```

Prevents invalid data.

Q26. What is Required Validation?

Example

```
email:{
 required:true
}
```

User must provide email.

Q27. What is Unique Validation?

Example

```
email:{
 unique:true
}
```

No duplicate emails.

Q28. What is Enum Validation?

Example

```
role:{
 enum:['admin','user']
}
```

Only allowed values.

Q29. What is Default Value?

Example

```
status:{
 default:'active'
}
```

Q30. What is Timestamp?

Example

```
{
 timestamps:true
}
```

Creates:

createdAt
updatedAt

Q31. What is Populate?

Answer

Populate works like SQL JOIN.

Example

User:

```
{
 orderId:ObjectId()
}
```

Populate fetches order details automatically.

Q32. Why Use Populate?

Answer

To fetch related documents.

Example:

User + Orders

in single query.

Q33. What is Indexing?

Answer

Indexes improve query performance.

Without Index:

Check every document

With Index:

Direct lookup

Much faster.

Q34. Example of Index?

```
UserSchema.index({
 email:1
})
```

Q35. What is Aggregation?

Answer

Used for complex data analysis.

Similar to SQL GROUP BY.

Q36. What is Aggregation Pipeline?

Example

```
[
 {$match:{}},
```

{$group:{}}
]

Processes data stage by stage.

Q37. What is $match?

Example

```
{
 $match:{
  work:'chef'
 }
}
```

Filters records.

Q38. What is $group?

Example

```
{
 $group:{
  _id:"$work",
  total:{
   $sum:1
  }
 }
}
```

Groups data.

Q39. What is $lookup?

Answer

Performs join between collections.

Example:

Users + Orders

Q40. What is Lean()?

Example

User.find().lean()

Returns plain JavaScript object.

Faster performance.

POPULATE SCENARIO

Question

One User has many Orders.

User Schema

```
orderIds:[
{
 type:mongoose.Schema.Types.ObjectId,
 ref:'Order'
}
]
```

Fetch Orders

```
User.find()
.populate('orderIds')
```

Interview Answer

Populate fetches complete order data using ObjectId references.

INDEXING SCENARIO

Question

Why is login API slow?

Problem

Searching email among 1 million users.

Without index:

Scan all records

Solution

```
UserSchema.index({
 email:1
})
```

Result

Fast login query.

AGGREGATION SCENARIO

Question

Count employees by role.

Data

```
Chef
Chef
Manager
Waiter
```

Query

```
Person.aggregate([
{
 $group:{
  _id:"$work",
  total:{
   $sum:1
  }
 }
}
])
```

Result

```
[
 {
  "_id":"chef",
  "total":2
 }
]
```

SCHEMA DESIGN SCENARIOS

Scenario 1

Restaurant App

User Schema

{
 name:String,
 email:String
}

Scenario 2

Menu Schema

{
 name:String,
 price:Number,
 taste:String
}

Scenario 3

Order Schema

{
 customerId:ObjectId,
 items:[]
}

Scenario 4

Hotel Management

Collections

Users
Menu
Orders

Tables
Payments

PRACTICAL TASKS (10)

Task 1

Create User Schema

name
email
age

Task 2

Create Model

mongoose.model()

Task 3

Insert User

await user.save()

Task 4

Find User By Email

```
User.findOne({
 email
})
```

Task 5

Update User

User.findByIdAndUpdate()

Task 6

Delete User

User.findByIdAndDelete()

Task 7

Add Validation

required:true

Task 8

Create Populate Relationship

ref:'Order'

Task 9

Create Index

```
UserSchema.index({
 email:1
})
```

Task 10

Write Aggregation Query

Count employees by role.

Use:

$group
$sum

CORE MERN QUESTIONS (1-20)

Q1. What is MERN Stack?

Answer

MERN is a full-stack JavaScript technology stack.

M = MongoDB

E = Express.js

R = React.js

N = Node.js

Flow

React (Frontend)
↓
Express API
↓
Node.js
↓
MongoDB

Q2. Why is MERN Popular?

Answer

Because entire application uses JavaScript.

Benefits:

- Single language
- Fast development
- Large community
- Easy API integration

Q3. What is React?

Answer

React is a JavaScript library used for building User Interfaces (UI).

Example

Facebook News Feed

Instagram Posts

Netflix Home Page

Q4. What is JSX?

Answer

JSX allows writing HTML inside JavaScript.

Example

```
const element = <h1>Hello Megha</h1>
```

Q5. What is a Component?

Answer

A component is a reusable UI block.

Example

```
function Header(){
 return <h1>Welcome</h1>
}
```

Q6. What is State?

Answer

State stores dynamic data inside a component.

Example

```
const [count,setCount] =
useState(0)
```

Q7. What is Props?

Answer

Props pass data from parent to child.

Example

```
<User name="Megha"/>
```

Q8. Difference Between State and Props?

State	Props
Mutable	Read Only
Internal	Parent Controlled

Q9. What is useState?

Answer

Hook used to manage state.

Example

```
const [name,setName] =
useState("")
```

Q10. What is useEffect?

Answer

Used for side effects.

Examples:

- API calls
- Timers
- Event listeners

```
useEffect(()=>{
 fetchUsers()
},[])
```

Q11. What is Virtual DOM?

Answer

React creates a virtual copy of DOM.

Updates only changed parts.

Result:

- Faster rendering

Q12. What is Axios?

Answer

Used to call APIs.

Example

```
axios.get('/users')
```

Q13. How Does React Connect to Backend?

Answer

Using HTTP requests.

```
axios.get(
 'http://localhost:3000/users'
)
```

Q14. What is Conditional Rendering?

Example

```
{
 isLoggedIn
 ?
 <Home/>
 :
 <Login/>
}
```

Q15. What is List Rendering?

Example

```
users.map(user=>(
 <li>{user.name}</li>
))
```

Q16. What is Controlled Component?

Example

```
<input
 value={name}
 onChange={handleChange}
/>
```

React controls input value.

Q17. What is Form Handling?

Answer

Managing user inputs using state.

Q18. What is React Router?

Answer

Used for navigation.

Example

```
<Route
 path="/home"
 element={<Home/>}
/>
```

Q19. What is SPA?

Answer

Single Page Application.

Page reload doesn't happen.

React updates content dynamically.

Q20. What is CORS?

Answer

Allows frontend and backend communication.

Backend:

app.use(cors())

ADVANCED MERN QUESTIONS (21-40)

Q21. What is Redux?

Answer

State management library.

Used when application becomes large.

Q22. Why Use Redux?

Problem

Passing props through many components.

Solution

Centralized state.

Q23. Redux Flow?

Component
↓
Dispatch
↓
Action
↓
Reducer
↓
Store
↓
Component

Q24. What is Store?

Answer

Central place where Redux stores data.

Q25. What is Action?

Answer

Describes what happened.

```
{
 type:"ADD_USER"
}
```

Q26. What is Reducer?

Answer

Updates state.

Q27. What is JWT Authentication?

Flow

Login
↓
Generate JWT
↓
Frontend Stores Token
↓
API Request
↓
Verify JWT

Q28. Where Should JWT Be Stored?

Common Answers

- Local Storage
- HTTP Only Cookies

Interviewers often ask this.

Q29. What is Protected Route?

Example

Only logged-in users can access:

/profile

Q30. How Do You Logout User?

Answer

Remove JWT token.

localStorage.removeItem()

Q31. Explain Full MERN Request Flow

Flow

React
↓
Axios Request
↓
Express Route
↓
Controller
↓
MongoDB
↓
Response
↓
React UI Update

Very Important.

Q32. What is MVC Architecture?

Structure

Models
Controllers
Routes

Q33. What is API Integration?

Example

Frontend calls backend API.

axios.get('/api/users')

Q34. What is Pagination?

Answer

Load records in chunks.

Example:

10 users per page.

Q35. What is Debouncing?

Example

Search Box

Instead of 100 API calls:

Wait 500ms then call API.

Q36. What is Lazy Loading?

Answer

Load components only when needed.

Improves performance.

Q37. What is Environment Variable?

Example

MONGO_URL=
JWT_SECRET=

Q38. What is Deployment?

Answer

Making app available online.

Frontend:

- Vercel
- Netlify

Backend:

- Render
- Railway

Q39. What Happens When User Clicks Login?

Interview Flow

Frontend
↓
POST /login
↓
Node
↓
MongoDB
↓
JWT
↓
Frontend

Very common interview question.

Q40. Explain Your MERN Project Architecture

Example

```
Frontend
 ├─ Components
 ├─ Pages
 └─ Services

Backend
 ├─ Routes
```

├─ Controllers
├─ Models
└─ Middleware

AUTHENTICATION SCENARIO

User Login

Frontend

axios.post('/login')

Backend

jwt.sign()

Result

Token Generated

Protected Route

Frontend

Authorization:
Bearer token

Backend

jwt.verify()

FULL STACK FLOW SCENARIO

Add Employee

Step 1

User fills form.

Name
Email
Role

Step 2

React sends POST request.

axios.post('/employees')

Step 3

Express receives request.

Step 4

MongoDB stores data.

Step 5

Response returned.

```
{
 success:true
}
```

PROJECT TASKS (10)

Task 1

Create Login Page

Requirements:

- Email
- Password
- Login Button

Task 2

Create Registration API

Fields:

Name
Email
Password

Store in MongoDB.

Task 3

Hash Password

Use:

bcrypt.hash()

Task 4

Generate JWT

Use:

jwt.sign()

Task 5

Create Protected Route

Use:

jwt.verify()

Task 6

Fetch Users in React

Use:

axios.get()

Display users.

Task 7

Create CRUD App

Features:

- Add User
- Edit User
- Delete User

- View Users

Task 8

Create Search Functionality

Use:

filter()

or backend search API.

Task 9

Create Pagination

Display:

10 Records/Page

Task 10

Build Employee Management System

Features

- Login
- Register
- Add Employee
- Edit Employee
- Delete Employee
- Search Employee
- JWT Authentication

CORE REACT QUESTIONS (1-20)

Q1. What is React.js?

Answer

React is a JavaScript library used to build user interfaces (UI).

Developed by:
Meta

React helps create:

- Single Page Applications
- Dynamic Websites
- Dashboards
- E-commerce Applications

Example

Applications using React:

- Facebook
- Instagram
- Netflix

Q2. Why Use React?

Answer

Benefits:

- Reusable Components
- Fast Rendering
- Virtual DOM
- Large Community

Example

Instead of creating navbar multiple times:

<Navbar />

Reuse anywhere.

Q3. What is a Component?

Answer

A component is a reusable piece of UI.

Example

```
function Header(){
 return <h1>Hello</h1>
}
```

Q4. Types of Components?

1. Functional Component

```
function App(){
 return <h1>Hello</h1>
}
```

2. Class Component

```
class App extends React.Component{
 render(){
  return <h1>Hello</h1>
 }
}
```

Today mostly Functional Components are used.

Q5. What is JSX?

Answer

JSX allows HTML inside JavaScript.

Example

```
const element =
<h1>Hello Megha</h1>
```

Q6. Why JSX?

Answer

Without JSX:

React.createElement()

Hard to write.

With JSX:

<h1>Hello</h1>

Easy to read.

Q7. What is Virtual DOM?

Answer

Virtual DOM is a lightweight copy of Real DOM.

Flow

State Change
↓
Virtual DOM
↓
Compare Changes
↓
Update Real DOM

This makes React fast.

Q8. What is Real DOM?

Answer

Actual browser DOM.

Updating Real DOM repeatedly is slow.

Q9. What is State?

Answer

State stores dynamic data.

Example

const [count,setCount]
=
useState(0)

Q10. What is Props?

Answer

Props pass data from Parent to Child.

Example

<User name="Megha"/>

Q11. Difference Between State and Props?

State	Props
Mutable	Read Only
Internal	Passed from Parent
useState	Component Attribute

Q12. What is useState?

Answer

Hook used to create state.

Example

```
const [name,setName]
=
useState("")
```

Q13. What is useEffect?

Answer

Used for side effects.

Examples:

- API Calls
- Timers
- Event Listeners

Example

```
useEffect(()=>{
 fetchData()
},[])
```

Q14. What Does [] Mean in useEffect?

Answer

Runs only once.

When component mounts.

Q15. What is Event Handling?

Example

```
<button
 onClick={handleClick}
>
Click
</button>
```

Q16. What is Conditional Rendering?

Example

```
{
 isLoggedIn
 ?
 <Home/>
 :
 <Login/>
}
```

Q17. What is List Rendering?

Example

```
users.map(user=>(
 <li>{user.name}</li>
))
```

Q18. Why is Key Required?

Answer

React identifies list items using keys.

<li key={user.id}>

Q19. What is Controlled Component?

Example

```
<input
 value={name}
 onChange={handleChange}
/>
```

React controls input.

Q20. What is Form Handling?

Answer

Managing form data using state.

ADVANCED REACT QUESTIONS (21-40)

Q21. What is React Router?

Answer

Used for navigation.

Example

```
<Route
 path="/home"
 element={<Home/>}
/>
```

Q22. What is BrowserRouter?

Answer

Wraps entire application.

<BrowserRouter>
<App/>
</BrowserRouter>

Q23. What is useNavigate?

Example

const navigate =
useNavigate()

navigate('/home')

Q24. What is useParams?

Example

URL:

/user/101

Access:

const {id}
=
useParams()

Q25. What is useRef?

Answer

Stores value without re-rendering.

Example

const inputRef =
useRef()

Q26. What is useMemo?

Answer

Caches expensive calculations.

Improves performance.

Q27. What is useCallback?

Answer

Prevents unnecessary function recreation.

Q28. What is React.memo?

Answer

Prevents unnecessary component re-rendering.

Q29. What is Context API?

Answer

Global state management.

Alternative to Redux for smaller apps.

Q30. What is Prop Drilling?

Answer

Passing props through many components.

Problem

App
↓
Parent
↓
Child
↓
GrandChild

Q31. What is Redux?

Answer

State management library.

Used in large applications.

Q32. Redux Flow?

Component
↓
Dispatch
↓
Action
↓
Reducer
↓
Store

Q33. What is Lazy Loading?

Example

```
const Home =
lazy(()=>import('./Home'))
```

Loads component when needed.

Q34. What is Code Splitting?

Answer

Breaking large bundle into smaller chunks.

Improves performance.

Q35. What is Debouncing?

Example

Search box waits 500ms before API call.

Reduces API requests.

Q36. What is Lifting State Up?

Answer

Moving state to common parent component.

Q37. What is Custom Hook?

Example

```
function useFetch(){
}
```

Reusable hook logic.

Q38. What is Error Boundary?

Answer

Catches UI errors.

Mostly used with Class Components.

Q39. What is Hydration?

Answer

Used in Server Side Rendering.

Commonly discussed with React frameworks.

Q40. How Does React Rendering Work?

Flow

State Change
↓
Virtual DOM
↓
Diffing
↓
Reconciliation
↓
Real DOM Update

Very important interview question.

HOOKS SCENARIOS

Scenario 1

Fetch Users

```
useEffect(()=>{
 axios.get('/users')
},[])
```

Scenario 2

Counter

```
const [count,setCount]
=
useState(0)
```

Scenario 3

Focus Input

```
const inputRef=
useRef()

inputRef.current.focus()
```

REACT ROUTER SCENARIOS

Route Setup

```
<Routes>

<Route
 path="/"
 element={<Home/>}
/>
```

```
<Route
 path="/about"
 element={<About/>}
/>

</Routes>
```

Dynamic Route

/user/:id

Example:

/user/101

STATE MANAGEMENT SCENARIO

Without Context

App
↓
Parent
↓
Child
↓
GrandChild

Prop drilling problem.

With Context

Context
↓
Any Component

Much easier.

REACT CODING TASKS (10)

Task 1

Create Counter App

Requirements:

- Increment Button
- Decrement Button

Use:

useState()

Task 2

Create Login Form

Fields:

Email
Password

Task 3

Fetch API Data

Use:

axios.get()

Display users.

Task 4

Create Search Filter

Use:

filter()

Task 5

Create Todo App

Features:

- Add Task
- Delete Task

Task 6

Create User List

Use:

map()

Task 7

Create Dynamic Route

/user/:id

Task 8

Create Protected Route

Only logged-in users can access dashboard.

Task 9

Create Theme Toggle

Light Mode

Dark Mode

Use:

useState()

Task 10

Create Employee CRUD App

Features:

- Add Employee

- Edit Employee
- Delete Employee
- Search Employee